Quantizing ESM-2 for Protein Language Modelling: A Measurement Study on Speed, Memory, and Accuracy

Qing Shao
Department of Chemical and Materials Engineering, University of Kentucky
Benchmarking report — ESM2-3B on NVIDIA H200 and A100

7 August 2026

Abstract

We benchmark six numerical precision configurations for ESM-2 protein language models across three axes — throughput, memory footprint, and predictive accuracy — on two workloads with sharply different characteristics: bulk embedding extraction and deep mutational scanning (DMS) variant-effect scoring. For ESM2-3B on an H200, FP8 dynamic quantization (`fp8_dyn`) delivers 1.15× the throughput of bf16 while halving peak memory (12.63 → 7.24 GB), making it the best configuration for embedding extraction. The same configuration is a poor choice for DMS scoring, where bf16 in eager mode is fastest at every problem size and quantization is pure overhead. Accuracy is evaluated on the complete ProteinGym substitution benchmark — 201 assays, 2.41M variants, both model scales — with a paired bootstrap clustered on protein. Two findings follow. First, fidelity measured *against fp32*, the metric such studies normally report, predicts the *magnitude* of the change in real-world accuracy ($r = 0.81$ over 1005 assay/configuration pairs) but not its *sign*: INT4 weight-only quantization is significantly *beneficial* at 3B and significantly *harmful* at 650M. Second, and of more practical consequence, no configuration shifts mean benchmark correlation by more than 0.007, yet INT8 dynamic quantization — indistinguishable from fp32 on that mean ($p = 0.34$) — takes a single assay from $\rho = 0.591$ to 0.223. Benchmark averages conceal precisely the failure that determines whether a configuration is deployable. We conclude that drift-from-fp32 metrics are sound risk bounds but unsound quality rankings, that configuration selection should be made on worst-case rather than mean behaviour, and we report four measurement artifacts encountered during this study, three of which inverted the result they were meant to measure.

1 Introduction

Quantization is usually presented as a straightforward trade: reduce numerical precision, gain speed and memory, lose a little accuracy. This framing is inherited from large autoregressive language models, where it is broadly correct. We set out to test whether it transfers to ESM-2, a family of protein language models, and found that essentially every part of it requires qualification.

ESM-2 is a BERT-style bidirectional *encoder*. It performs a single forward pass with no KV cache and no autoregressive decode. This places it in a **compute-bound** regime rather than the memory-bandwidth-bound regime that governs LLM inference at batch size 1. The distinction is not academic: the dominant open-source quantization toolchains (GPTQ, AWQ, bitsandbytes NF4, GGUF) are all *weight-only*. They store INT4/INT8 weights and dequantize to bf16 immediately

before an ordinary bf16 matrix multiply. When the bottleneck is memory bandwidth, this is a large win. When the bottleneck is arithmetic, as it is here, it adds work to an already-saturated kernel.

This report addresses three questions:

1. What do the available quantization configurations actually cost and deliver on the three axes of speed, memory, and accuracy?
2. Does the answer depend on the workload? (It does, and the two workloads we study prefer opposite configurations.)
3. Do the fidelity metrics conventionally used to validate quantization predict real-world predictive accuracy? (Only partially, and in a way that matters.)

2 Experimental Design

2.1 Configurations

Six configurations were evaluated, all applied via `torchao` to the encoder `Linear` layers only:

Configuration	Class	Description
`fp32`	baseline	Reference for all fidelity measurements.
`bf16`	baseline	The practical production default.
`int8_wo`	weight-only	INT8 weights, bf16 compute. Memory-oriented.
`int4_wo`	weight-only	INT4 weights, bf16 compute. Smallest footprint.
`int8_dyn`	W8A8	INT8 weights *and* dynamically quantized activations; uses INT8 tensor cores.
`fp8_dyn`	W8A8	FP8 (E4M3) weights and activations. Requires compute capability ≥ 8.9; H200 only.

Table 1: Quantization configurations. Weight-only variants target memory; the dynamic (W8A8) variants also quantize activations and can therefore use low-precision tensor cores for the matrix multiply itself.

2.2 Workloads

Bulk embedding extraction. Large length-bucketed batches under a token budget, measured in residues per second. This is the compute-bound regime.

DMS variant-effect scoring. The standard ESM masked-marginals protocol: for each mutated position, mask it and record $\log p(\text{mut}) - \log p(\text{wt})$ from the resulting distribution. Cost scales with the *number of mutated positions*, not with the number of variants, and the masked batch is narrow. This is the latency-bound regime.

Two optimizations were applied to the DMS path before benchmarking, both of which are absent from typical reference implementations: only positions carrying a variant are masked, and the masked copies are batched rather than run one at a time.

2.3 Metrics

Three fidelity metrics were recorded, deliberately ordered by sensitivity:

1. **Embedding cosine similarity** (mean-pooled, excluding special tokens and padding). Forgiving: error averages out along the sequence.

2. **Logit KL divergence**, per residue. Moderately sensitive.

3. **DMS Spearman ρ against fp32.** Most sensitive, because the score is a *difference of two near-equal log-probabilities*, so quantization noise does not cancel and is amplified relative to signal.

All three are drift-from-fp32 measures. They quantify how far a configuration moves the answer, not whether it moves toward or away from the truth. To measure the latter, we added a fourth metric requiring real experimental data: **Spearman ρ against wet-lab DMS measurements**, from ProteinGym.

2.4 Real assay data

Three ProteinGym substitution assays were selected to span the sequence-length range, since peak memory during masked-marginals scoring grows with L:

Assay	L	Singles	Masked positions	Coverage
`IF1_ECOLI_Kelsic_2016`	72	1,367	72	100%
`BLAT_ECOLX_Stiffler_2015`	286	4,996	263	92%
`HSP82_YEAST_Flynn_2019`	709	13,194	707	100%

Table 2: ProteinGym assays used. All 19,557 single-substitution variants had their stated wild-type residue verified against the reference `target_seq` before use; all matched. This check is load-bearing — an off-by-one in position indexing does not raise an error, it silently scores the wrong positions and yields a plausible-looking correlation.

2.5 Full benchmark data

The three-assay analysis is superseded by a run over the complete ProteinGym substitution benchmark. All 217 assays were extracted from the `ProteinGym_v1` parquet release and every one passed wild-type validation across 2,465,767 variants. The 201 assays fitting ESM-2's 1022-residue context were scored: 2,413,913 variants over 40,775 masked positions, spanning $L = 37$ to 934 and five function categories.

The masked-position count, not the variant count, sets the cost: masked-marginals scoring performs one forward pass per mutated position, so 2.41M variants are nearly free on top of 40,775 forwards.

An earlier three-assay path combined per-assay CSVs from the `v0.1` release with wild-type sequences from the `v1.0` reference file. The two agree on the 85 assays present in both and diverge elsewhere, including renames. That is acceptable for three hand-picked assays and unsafe at benchmark scale; the `v1` parquet carries `target_seq` inline, so scores and wild-type sequences come from a single release.

2.6 Statistical treatment

Ground-truth correlations are compared using a **paired bootstrap** (2000 resamples). Both the candidate configuration and the fp32 reference are scored on the *same* resample at each iteration, so shared noise cancels and the residual is attributable to the configuration. The pairing is not a

refinement: for `int4_wo` the paired interval is 8.4× narrower than the unpaired one, because per-assay ρ spans 0.0–0.9 while the configuration effect is of order 0.006. Unpaired, every configuration reads as noise.

The *resampling unit* differs by scale of claim. For a single assay the unit is the variant, which establishes that a shift is not an artifact of the variant set. For the full benchmark the unit is the assay, since the claim becomes one about assays in general. Because the 201 assays cover only 173 distinct proteins, benchmark-level intervals use a **cluster bootstrap over proteins**: whole proteins are drawn and all their assays retained, so correlated replicates cannot count as independent evidence. Clustered and unclustered intervals agreed to the fourth decimal, which is reported as a measured result rather than assumed.

Intervals are percentile bootstrap. They were checked against a normal approximation ($\bar{d} \pm 1.96\,\mathrm{SE}$) and agree to four decimals on every configuration, confirming the underlying distribution is near-symmetric and the percentile method is not distorting the interval. The procedure was validated on synthetic data with a known answer: a negligible perturbation was correctly classified as noise, a genuine improvement as real.

2.7 Hardware and software environment

Component	Detail
GPU (primary)	NVIDIA H200, 143 GB, sm_90 (FP8 tensor cores)
GPU (secondary)	NVIDIA A100, 80 GB, sm_80 (no FP8)
Driver	550.54.14
PyTorch	2.6.0+cu124
Attention	SDPA
Compile mode	`max-autotune-no-cudagraphs`

The cluster runs CentOS 7.6 with glibc 2.17 on every node, including the H200 nodes, which forced several choices. PyTorch moved to `manylinux_2_28` at version 2.7, making **2.6.0 the newest installable release**. `bitsandbytes` GPU wheels require a newer glibc, so only the CPU-only 0.42.0 installs and its NF4 path is unavailable — 4-bit quantization therefore goes through `torchao`. The system GCC (4.8.5) cannot build Triton's helper module, so `torch.compile` fails until `CC`/`CXX` are pointed at GCC 12.1.0. Finally, `$HOME` and `/project` are at quota, and the Triton, Inductor, HuggingFace, and pip caches all default there; all were relocated to `/scratch`. `torchao` nonetheless works because its INT8/FP8 paths dispatch to `torch._int_mm` and `torch._scaled_mm`, which live inside PyTorch and support sm_90.

3 Results

3.1 Compile mode determines whether W8A8 is viable at all

The single most consequential configuration choice is not the quantization scheme but the `torch.compile` mode. Measured on a single ESM2-3B-shaped feed-forward block, $2560 \to 10240 \to 2560$, at batch 32×512 on an A100:

Measured in eager mode, W8A8 appears 4.2× *slower* than bf16 and would be discarded. Measured under `max-autotune` it is 1.46× faster. `dynamic=True` was avoided because it emits shape-generic kernels that surrender most of the gain; static shapes are kept manageable by padding to a fixed multiple of 128. `torch._dynamo.explain` confirms **zero graph breaks** for both bf16 and `int8_dyn`, establishing this as a kernel-selection effect rather than a Dynamo fallback.

Variant	Time (ms)
bf16 eager	7.91
bf16 compiled	7.91
`int8_dyn` eager	33.35
`int8_dyn` compiled, `dynamic=True`	11.73
`int8_dyn` compiled, default mode	11.48
`int8_dyn` `mode="max-autotune"`	**5.43**

Table 3: Only `max-autotune` allows Inductor to autotune the INT8 Triton matmuls, which is where the entire W8A8 benefit resides.

3.2 Bulk embedding extraction

Config	Weights (GB)	Peak (GB)	Residues/s	vs bf16	Emb. cos	Logit KL	DMS ρ_{fp32}
`fp32` (ref)	10.72	25.37	6,750	0.12×	—	—	—
`bf16`	5.29	12.63	57,057	1.00×	0.999949	1.17e–04	0.9998
`int8_wo`	2.66	8.48	54,816	0.96×	0.999959	1.05e–04	0.9998
`int8_dyn`	2.65	**7.07**	59,996	1.05×	0.989592	1.40e–02	0.9928
`fp8_dyn`	2.65	7.24	**65,376**	**1.15**×	0.999920	4.60e–04	0.9988
`int4_wo`	**1.61**	7.69	3,813	0.07×	0.998904	1.89e–03	0.9975

Table 4: ESM2-3B, H200, compiled with `max-autotune-no-cudagraphs`, SDPA attention, 64 sequences of length 512–1024 under a 16,384-token budget. Reproduced across four independent jobs: bf16 mean 56,642 residues/s, range 55,941–57,422 (±1.3%); fp32 mean 6,742, range 6,687–6,814.

Three observations. First, the **fp32 → bf16 step is worth** 8.4× — larger than everything quantization adds on top of it, and the single biggest lever available. Second, `fp8_dyn` dominates `int8_dyn`: it is faster (1.15× vs 1.05×) *and* dramatically more accurate (embedding cosine 0.999920 vs 0.989592; logit KL 4.6×10^{-4} vs 1.4×10^{-2}), surrendering only 0.17 GB of peak memory. FP8's E4M3 format has far greater dynamic range than INT8 at the same bit width, which matters because transformer activations contain outliers. Third, `int4_wo` is a trap for inference: 0.07× throughput and *worse* peak memory (7.69 GB) than `fp8_dyn` (7.24 GB) despite 40% smaller weights, because the dequantization path allocates transient buffers. Its value lies elsewhere, as a QLoRA base.

Separately, length-bucketed batching reduced padding waste from 25.0% (naive fixed-size batching) to **0.0%**. This is free and costs no accuracy — a larger effect than several of the quantization differences in the table.

3.3 DMS scoring: the opposite conclusion

bf16 is fastest at every assay size, and `int8_dyn` — the second-fastest configuration for bulk extraction — is up to 17× slower here. The masked batch (n_positions × L) is too small to contain a matrix multiply large enough for low precision to pay for its quantize/dequantize overhead.

Notably, `fp8_dyn` closes on bf16 as the assay grows: 0.26×, 0.50×, 0.81× of bf16 throughput at $L = 72, 286, 709$ respectively. Its overhead is fixed per call while real work scales, so at $L = 709$ it costs 20% throughput for 43% less memory — a defensible trade that is indefensible at $L = 72$.

Config	Variants/s		
	IF1 (L=72)	BLAT (L=286)	HSP82 (L=709)
`bf16`	**6,979**	**3,573**	**1,357**
`int8_wo`	4,830	2,511	1,066
`fp8_dyn`	1,813	1,796	1,103
`fp32`	1,460	447	187
`int8_dyn`	410	382	299
`int4_wo`	1,045	282	112

Table 5: DMS scoring throughput, ESM2-3B, uncompiled. bf16 wins at every problem size; quantization is pure overhead in this regime.

Config	Weights (GB)	Peak L=72	Peak L=286	Peak L=709
`fp32`	10.72	10.87	11.19	11.85
`bf16`	5.29	5.38	5.55	5.88
`int8_wo`	2.66	2.77	2.92	3.25
`int8_dyn`	2.65	2.87	2.91	3.36
`fp8_dyn`	2.65	2.76	2.96	3.35
`int4_wo`	**1.61**	**1.70**	**1.87**	**2.20**

Table 6: Peak device memory during DMS scoring. Weights account for approximately 90% of peak even at $L = 709$ (bf16: 5.29 GB weights against 0.59 GB activations).

3.4 Memory during DMS scoring

This is the reverse of the bulk-extraction situation. Masked-marginals scoring keeps the batch narrow, so the $O(L^2)$ attention term never dominates and weights remain roughly 90% of peak memory even at $L = 709$. Quantization therefore addresses almost all of DMS peak memory, and `int4_wo` gives the smallest footprint at every length — 2.7× below bf16.

3.5 Ground-truth accuracy on ProteinGym

Two findings emerge, and the second qualifies the first.

Quantization does not systematically degrade variant-effect accuracy. Of the 15 assay/configuration pairs, 11 shifts are upward. Most are statistically indistinguishable from zero. Strikingly, `int4_wo` — the worst configuration on every fidelity metric — produces the *best* agreement with experiment on BLAT (+0.0495, CI $[+0.0433, +0.0561]$). This survives the bootstrap and is not a resampling artifact.

Fidelity-vs-fp32 predicts the magnitude of the shift, not its sign. Across all 15 pairs, the correlation between fidelity loss $(1 - \rho_{\text{fp32}})$ and the absolute ground-truth shift is $r = 0.87$ (Pearson), $\rho = 0.76$ (Spearman). But `int4_wo` moves +0.0495 on BLAT and −0.0167 on IF1. The direction is a property of the assay, not of the configuration, and is not predictable from any drift-from-fp32 measurement.

The practical consequence is that `bf16` and `int8_wo` shift ground-truth agreement by at most 0.002 on any assay tested — below anything an experiment could resolve — and are therefore safe. `int4_wo` is a ±0.05 gamble on an unknown sign.

Assay	Config	ρ_{expt}	Δ vs fp32	95% CI of Δ	Verdict
IF1	`fp32`	0.5561	(ref)		
(n=1367)	`bf16`	0.5543	−0.0018	[−0.0027, −0.0009]	**real**
	`int8_wo`	0.5570	+0.0009	[−0.0003, +0.0021]	noise
	`int8_dyn`	0.5504	−0.0057	[−0.0124, +0.0013]	noise
	`fp8_dyn`	0.5584	+0.0023	[−0.0022, +0.0067]	noise
	`int4_wo`	0.5394	**−0.0167**	[−0.0263, −0.0074]	**real**
BLAT	`fp32`	0.5893	(ref)		
(n=4996)	`bf16`	0.5896	+0.0003	[−0.0003, +0.0009]	noise
	`int8_wo`	0.5913	+0.0019	[+0.0011, +0.0029]	**real**
	`int8_dyn`	0.6160	**+0.0266**	[+0.0219, +0.0315]	**real**
	`fp8_dyn`	0.6088	**+0.0194**	[+0.0161, +0.0229]	**real**
	`int4_wo`	0.6388	**+0.0495**	[+0.0433, +0.0561]	**real**
HSP82	`fp32`	0.2931	(ref)		
(n=13194)	`bf16`	0.2933	+0.0002	[−0.0002, +0.0005]	noise
	`int8_wo`	0.2933	+0.0002	[−0.0002, +0.0007]	noise
	`int8_dyn`	0.2937	+0.0006	[−0.0020, +0.0028]	noise
	`fp8_dyn`	0.2944	+0.0012	[−0.0003, +0.0027]	noise
	`int4_wo`	0.2904	−0.0027	[−0.0057, +0.0002]	noise

Table 7: Spearman correlation against experimental measurements, ESM2-3B. Δ is the change from fp32, with a 2000-sample paired bootstrap over variants. "Real" indicates the 95% confidence interval excludes zero.

This supersedes an earlier conclusion drawn from a synthetic 40-residue mutational scan, which ranked `int8_dyn` ($\rho_{\text{fp32}} = 0.9928$) as risky and `fp8_dyn` (0.9988) as clearly safer. Both numbers were correct; neither predicted experimental accuracy. On BLAT the "risky" configuration gained +0.027.

We note that the assay itself matters far more than any quantization decision: ρ_{expt} ranges from 0.29 (HSP82) to 0.59 (BLAT) at 3B. Choosing which assay to trust is a 2× effect; choosing a quantization configuration is a 1% effect.

3.6 Model-scale consistency

The same three assays were run on ESM2-650M. The qualitative picture holds: bf16 fastest, `int8_dyn` slowest, ground-truth shifts small. The largest 650M shift was −0.0063 (`int8_dyn` on IF1) against ρ_{fp32} of 0.9954. The larger and more clearly significant shifts at 3B indicate that sensitivity to quantization grows with model size, so 650M results should not be extrapolated to 3B without checking.

An incidental observation deserves note, because it bears directly on how much any of this matters. The fp32 650M model correlates with experiment *better* than the fp32 3B model on two of the three assays:

Assay	650M ρ_{expt}	3B ρ_{expt}	Difference
IF1	0.5987	0.5561	−0.0426
BLAT	0.7315	0.5893	−0.1422
HSP82	0.2648	0.2931	+0.0283

The BLAT gap of 0.14 is roughly *three times* the largest effect any quantization configuration

produced in this study. Model selection dominates precision selection by a wide margin, and the assumption that the larger checkpoint is the more accurate one does not hold uniformly here.

This inference does not survive Section 3.7. Over 201 assays the two models are statistically indistinguishable ($p = 0.35$). The BLAT gap is real for BLAT and does not generalise — the same error, made the same way, as the `int4_wo` result three subsections above.

3.7 The full benchmark: 201 assays

Every accuracy conclusion above rests on three assays. Three is enough to establish that an effect exists on a particular assay and not enough to establish anything about assays in general, because the unit that varies — the assay — is held fixed. The whole ProteinGym substitution benchmark was therefore run: all 217 assays validated, the 201 within ESM-2's 1022-residue context scored, 2,413,913 variants over 40,775 masked positions, six configurations, two model scales. 3.1 GPU-hours at 3B.

Two changes to the statistical treatment follow from the larger sample. The resampling unit becomes the **assay** rather than the variant, since the question is now whether a configuration degrades prediction in general. And because the 201 assays cover only 173 distinct proteins — `BLAT_ECOLX` appears four times — the reported interval is a **cluster bootstrap over proteins**, which prevents correlated replicates from counting as independent evidence. The two intervals turned out to agree to the fourth decimal, so the clustering was immaterial here; it is reported because that had to be measured rather than assumed.

	ESM2-3B			ESM2-650M		
Config	$\bar{\rho}$	Δ	95% CI	$\bar{\rho}$	Δ	95% CI
fp32	0.4398	—	(ref)	0.4459	—	(ref)
`bf16`	0.4399	+0.0002	$[-0.0001, +0.0004]$	0.4460	+0.0000	$[-0.0003, +0.0004]$
`int8_wo`	0.4399	+0.0001	$[-0.0002, +0.0005]$	0.4457	−0.0002	$[-0.0006, +0.0002]$
`fp8_dyn`	0.4403	+0.0005	$[-0.0007, +0.0017]$	0.4452	−0.0008	$[-0.0020, +0.0005]$
`int8_dyn`	0.4376	−0.0021	$[-0.0072, +0.0017]$	0.4440	**−0.0020**	$[-0.0032, -0.0009]$
`int4_wo`	0.4435	**+0.0037**	$[+0.0007, +0.0068]$	0.4395	**−0.0064**	$[-0.0105, -0.0018]$

Table 8: Mean Spearman against experiment over 201 assays. Bold marks intervals excluding zero under the protein-clustered paired bootstrap. Note that `int4_wo` is significant in *opposite directions* on the two model scales.

The benchmark mean is the wrong safety criterion

Every Δ in that table is below 0.007. The per-assay tail is not.

`int8_dyn` at 3B is indistinguishable from fp32 on the benchmark mean ($\Delta = -0.0021$, $p = 0.34$) while destroying `UBR5_HUMAN_Tsuboyama_2023_1I2T`, where correlation with experiment falls from 0.591 to 0.223 at $\rho_{\text{fp32}} = 0.393$. A configuration can pass the benchmark average and be unusable on the one protein a given project cares about. This is the practical result of the whole study: benchmark-mean significance testing answers a question about populations, and deployment is a question about a specific instance.

What the larger sample changed

The three-assay verdict was backwards. Section 3.5 concluded that quantization does not systematically degrade variant-effect accuracy, on the evidence that 11 of 15 shifts were upward

	ESM2-3B		ESM2-650M	
Config	worst Δ	$\lvert\Delta\rvert > 0.05$	worst Δ	$\lvert\Delta\rvert > 0.05$
`bf16`	−0.0072	0	−0.0040	0
`int8_wo`	−0.0106	0	−0.0120	0
`fp8_dyn`	−0.0344	0	−0.0316	0
`int8_dyn`	**−0.3688**	6	−0.0411	0
`int4_wo`	−0.0556	5	**−0.2024**	6

Table 9: Worst single-assay change in ground-truth correlation, and count of assays moving by more than 0.05, out of 201.

and that `int4_wo` gained +0.0495 on BLAT. At benchmark scale `int4_wo` is significant in opposite directions on the two model scales. Both measurements were correct; the generalisation from three assays was not.

Fidelity predicts magnitude, not direction. Across 1005 assay/configuration pairs, $r(1 - \rho_{\text{fp32}}, |\Delta|) = 0.81$ at 3B and 0.74 at 650M, confirming the 0.87 estimated from 15 pairs. The *signed* correlation is −0.58 at 3B and +0.10 at 650M — inconsistent in sign between model scales, and therefore useless for prediction. ρ_{fp32} remains the correct cheap screen, bounding how far an answer can move while saying nothing about which way.

Damage concentrates where the model is weakest. At 650M, `int4_wo` costs 0.0174 on low-MSA-depth assays against 0.0042 on high-depth ones. Quantization does most harm where there is least signal to lose — a breakdown three assays cannot support.

Speed and memory over the whole benchmark

Config	Wall-clock	Positions/s	Peak RAM (median)	Peak RAM (max)
fp32	45.0 min	25.6	11.10 GB	12.20 GB
`bf16`	**7.3 min**	**200.6**	5.50 GB	6.05 GB
`int8_wo`	9.2 min	139.3	2.87 GB	3.42 GB
`fp8_dyn`	10.2 min	89.2	2.90 GB	3.56 GB
`int8_dyn`	40.0 min	20.1	2.94 GB	3.51 GB
`int4_wo`	73.8 min	17.3	**1.82 GB**	**2.37 GB**

Table 10: Full 201-assay benchmark, ESM2-3B on one H200, uncompiled. Wall-clock is summed scoring time over all assays.

The DMS recommendation from three assays survives and strengthens: `bf16` eager, 6.2× faster than fp32 end-to-end, with ground-truth agreement never moving more than 0.018 across 402 assay/model pairs. `int8_wo` is the memory fallback at 69% of bf16 speed and maximum drift 0.016. `int8_dyn` and `int4_wo` are not trade-offs for this workload — they are slower than bf16 *and* carry a catastrophic tail.

4 Measurement Artifacts

Four measurement defects were found and corrected during this study. Three of them inverted or corrupted the very quantity being measured, and we document them because each is a general hazard rather than a local mistake.

4.1 Compile cost inside the timed region (three occurrences)

The DMS throughput column initially ranked `fp32` fastest and `fp8_dyn` slowest — an exact inversion of the bulk-throughput column. Two successive fixes failed. Direct instrumentation finally established the cause. On 650M/A100 with a 40-residue wild type over 11 masked positions:

Measurement	Time
eager, one forward at shape $(11, 42)$	27.9 ms
eager, full `masked_marginals` pass	0.03 s
compiled, one forward at shape $(11, 42)$	**15,138.1 ms**

DMS scoring uses a much smaller input shape than the bulk-throughput batches, and under `max-autotune` each new shape costs roughly 15 s to autotune. At this batch size that cost recurs rather than amortizing. The timed region was therefore measuring *autotune cost*, and ranking configurations by their number of Triton kernel candidates — `fp32` had the fewest and appeared fastest, `fp8_dyn` had 98 and appeared slowest. The fix was to score DMS against the uncompiled module.

This has a consequence beyond the benchmark: for short wild-type sequences with few mutated positions, `max-autotune` is actively counterproductive in production, not merely in measurement.

The same class of defect appeared earlier in the bulk-throughput path, where warming up on only a prefix of the batches left recompilation for unseen length buckets inside the timed region.

4.2 Peak memory attributed to the wrong model

Peak memory during DMS scoring was reported as follows, with the activation overhead implied by subtracting weights:

Config	Weights (GB)	Peak (GB)	Implied overhead
`fp32`	2.49	2.56	+0.07
`bf16`	1.21	3.74	+2.53 ≈ fp32's weights
`int8_wo`	0.61	1.86	+1.25 ≈ bf16's weights
`int8_dyn`	0.61	1.26	+0.65 ≈ int8_wo's weights

Each configuration's peak included the *previous* configuration's weights, producing the visible absurdity of bf16 requiring more peak memory than fp32. The cleanup helper deleted only its own local reference to the model; the caller retained a binding. Because `nn.Module` graphs contain reference cycles, dropping the last reference does not free them by reference counting alone — they persist as cyclic garbage until a collection runs. Inserting an explicit `gc.collect()` before each peak-memory reset corrected it; overhead became a consistent 0.04–0.07 GB.

The bulk-throughput memory column was verified unaffected: bf16's measured overhead there was +7.34 GB, which cannot contain fp32's 10.72 GB of weights.

4.3 Silent argument leakage in the job script

A batch script used `shift 3 || true`. When fewer than three positional arguments are supplied, `shift 3` fails, `|| true` suppresses the failure, and the arguments remain in `"$@"` — where they were appended to the Python invocation as stray arguments. Relying on a default for the third argument was sufficient to trigger it.

4.4 Common thread

Three of the four defects shared a signature: a cost that belonged outside the measurement leaked inside it, and the resulting ranking was plausible enough to be believed. In each case the tell was a result that inverted or contradicted a neighbouring measurement. We would emphasise that a benchmark producing a *clean, monotonic, plausible* table is not thereby correct; the memory bug produced exactly such a table for several runs.

5 Discussion

5.1 The two workloads want opposite configurations

This is the principal practical finding.

Bulk embedding extraction is compute-bound: large batches, large GEMMs. Quantization pays. `fp8_dyn` with `max-autotune` wins on all three axes — 1.15× bf16 throughput, weights 5.29 → 2.65 GB, peak 12.63 → 7.24 GB, with accuracy indistinguishable from bf16.

DMS variant-effect scoring is latency-bound: the masked batch is small, no GEMM is large enough for low precision to help, and per-linear quantize/dequantize overhead is pure loss. bf16 in eager mode is fastest at every problem size, and `max-autotune` is actively harmful.

Applying the embedding-optimized configuration to DMS scoring makes it up to an order of magnitude slower. Because both workloads commonly appear in the same pipeline, this is a live risk rather than a theoretical one.

5.2 On the interpretation of fidelity metrics

The conventional validation practice — quantize, measure drift against the full-precision model, accept if drift is small — is sound as a *risk bound*. Our data support the bound at scale: over 1005 assay/ configuration pairs, fidelity loss correlates with the magnitude of ground-truth change at $r = 0.81$ (3B) and 0.74 (650M), consistent with the 0.87 first estimated from 15 pairs.

It is not sound as a *quality ranking*. A configuration with larger drift is not thereby less accurate against reality; it is merely less predictable. The signed correlation is -0.58 at 3B and $+0.10$ at 650M — it does not even hold its sign between model scales.

A tempting misreading of our BLAT result is that quantization acts as beneficial regularization. We do not endorse this, and the full benchmark settles it: `int4_wo` is significantly *positive* at 3B and significantly *negative* at 650M. A configuration cannot be genuinely more accurate than the reference from which it is derived in any way one could rely on in advance.

5.3 Averages conceal the failure that matters

The benchmark-mean result is that no configuration changes mean ground-truth correlation by more than 0.007. Read alone, that licenses any configuration on the list. It should not.

`int8_dyn` at 3B is statistically indistinguishable from fp32 on the mean ($p = 0.34$) and takes one assay from $\rho = 0.591$ to 0.223. A practitioner does not deploy against the mean of 201 assays; they deploy against one protein. The benchmark mean answers whether a configuration is safe *on average over targets*, which is a claim about a population, while the operative question is whether it is safe on a specific target.

The practical consequence is that the tail statistics — worst-case single-assay drift, and the count of assays exceeding a tolerance — are the numbers to select on, and they separate the configurations far more sharply than the means do. On that criterion `bf16` and `int8_wo` are safe (no assay beyond

0.018 across 402 assay/model pairs), `fp8_dyn` is nearly so (0.034), and `int8_dyn` and `int4_wo` are not.

5.4 Recommendations

Situation	Recommendation
Bulk embedding extraction (H200)	`fp8_dyn` + `max-autotune-no-cudagraphs`
Bulk embedding extraction (A100)	`int8_wo`; `int8_dyn` costs real accuracy
DMS / variant effect scoring	`bf16`, eager, no compile
DMS under memory pressure	`int8_wo` (2.87 vs 5.50 GB; 69% of bf16 speed)
Minimum memory footprint	`int4_wo`, only if a -0.20 single-assay collapse is acceptable; validate on your target
QLoRA fine-tuning base	`int4_wo` (1.61 GB weights)
V100 / sm_70 hardware	Quantization buys memory only, never speed

Two non-quantization measures outweigh most of the above. Moving from fp32 to bf16 is worth 8.4× on bulk extraction and 6.2× over the full DMS benchmark; length-bucketed batching eliminates 25% of wasted computation. Both should be applied before any quantization is considered.

A third, proposed in an earlier draft of this report, does not survive the full benchmark: the 650M checkpoint beat 3B by 0.14 Spearman on BLAT, which suggested model selection dominated precision selection. Over 201 assays the two are indistinguishable (0.4459 vs 0.4398, 95% CI $[-0.0061, +0.0185]$, $p = 0.35$, with 650M ahead on 110 of 201). What remains true is narrower and still useful: the *uncertainty* on model choice (± 0.012) exceeds every quantization effect measured here (≤ 0.007). Precision is the settled variable; checkpoint choice is not, and is worth benchmarking on a representative assay.

6 Limitations

- **We still cannot predict which assays behave which way.** 201 assays establish the population behaviour of each configuration and the size of its tail. They do not identify in advance which target will be the one that collapses. The only reliable procedure remains scoring a candidate configuration against fp32 on the actual target.
- **The 16 longest assays are excluded.** Assays beyond 1022 residues exceed ESM-2's context and were skipped rather than truncated, since a truncation window is a scoring-protocol change that would confound the comparison. Configuration behaviour above $L = 934$ is therefore unmeasured.
- **Multiple comparisons are uncorrected.** Five configurations are tested against fp32 per model. Both significant results survive a Bonferroni threshold of $0.05/5 = 0.01$, but `int4_wo` at 650M sits exactly on it ($p = 0.010$).
- **Bootstrap resolution.** At 2000 resamples the smallest reportable two-sided p is 0.001; values at that floor should be read as $p < 0.002$ rather than as point estimates.
- **Low-signal assays resolve nothing individually.** Where $\rho_{\text{expt}} \approx 0.3$ the model's own error dominates and configuration differences fall below the noise floor; such assays contribute to the benchmark mean but carry little information alone.

- **Sequence-length coverage.** Bulk-throughput measurements used sequences of 512–1024 residues. Beyond $L \approx 2000$ the $O(L^2)$ attention term dominates peak memory and quantization's share of the memory benefit shrinks correspondingly. A representative FASTA from the target workload would settle this.
- **Single random seed** for quantization; no ensemble over calibration variation.
- **The QLoRA track is unbuilt.** `int4_wo` at 1.61 GB is the natural base, and fine-tuning is the one setting where quantization buys capability rather than efficiency — full 3B fine-tuning requires roughly 45 GB of optimizer state alone.

7 Conclusions

1. For **bulk embedding extraction** of ESM2-3B on H200 hardware, `fp8_dyn` under `max-autotune` is the best configuration on all three axes at once: 1.15× bf16 throughput, 43% less peak memory, and no measurable accuracy cost.
2. For **DMS variant-effect scoring**, quantization is counterproductive. bf16 in eager mode is fastest at every problem size, and compilation is actively harmful. The two workloads want opposite configurations.
3. **Drift from fp32 bounds risk; it does not rank quality.** Over 1005 assay/configuration pairs it predicts the magnitude of real-world accuracy change ($r = 0.81$) and not its sign, which does not even hold between model scales. Configurations whose drift is small enough that the sign cannot matter — `bf16` and `int8_wo`, never beyond 0.018 Spearman on any of 402 assay/model pairs — are the defensible choices when accuracy is at stake.
4. **Select on the tail, not the mean.** No configuration moves mean benchmark correlation by more than 0.007, and that fact is nearly useless for deployment. `int8_dyn` passes the mean test at 3B ($p = 0.34$) while taking one assay from $\rho = 0.591$ to 0.223. Worst-case single-assay drift separates the configurations by two orders of magnitude where the means separate them by none.
5. **Validate on the workload you actually run, at the scale you intend to generalise to.** A synthetic mutational scan produced a confident and incorrect ranking of configurations. Three real assays corrected it and produced a second incorrect generalisation — that quantization does not systematically degrade accuracy, and that checkpoint choice dominates precision choice. Both fell to 201 assays. Each step was a genuine improvement in evidence and each intermediate conclusion was wrong in a way that looked well-supported at the time.

A Reproduction

```
# Environment (handles glibc/GCC/cache-quota constraints)
source env/activate.sh

# Fetch assay data; verifies variant indexing against target_seq
python src/fetch_proteingym.py IF1_ECOLI_Kelsic_2016 \
    BLAT_ECOLX_Stiffler_2015 HSP82_YEAST_Flynn_2019

# Bulk throughput + fidelity matrix (compiled)
sbatch -p H8V141_SAP112M2000_L --mem=200G slurm/run_matrix.sbatch \
```

```
    3B fp32,bf16,int8_wo,int8_dyn,fp8_dyn,int4_wo --compile

# Real-assay DMS: accuracy, speed, memory (uncompiled by design)
sbatch -p H8V141_SAP112M2000_L --mem=200G slurm/run_dms.sbatch \
    3B fp32,bf16,int8_wo,int8_dyn,fp8_dyn,int4_wo

# Significance of ground-truth shifts (single assay, variant bootstrap)
python src/analyze_dms.py results/dms_3B_*_<jobid>_scores.json

# --- Full benchmark ----------------------------------------------------
# All 217 assays; validates every variant's WT and refuses to write on
# mismatch.  ~110 MB of parquet, cached in data/proteingym_v1/_raw
python src/fetch_proteingym_v1.py

# One array task per config; each loads its model once and scores all
# 201 assays.  Resumable: a resubmission skips assays already written.
sbatch --array=0-5 -p H8V141_SAP112M2000_L --mem=200G \
    slurm/run_proteingym.sbatch 3B
sbatch --array=0-5 -p H8V141_SAP112M2000_L --mem=200G \
    slurm/run_proteingym.sbatch 650M

# Benchmark-wide accuracy / speed / RAM + protein-clustered bootstrap
python src/aggregate_proteingym.py --model 3B
python src/aggregate_proteingym.py --model 650M
```

Result artifacts referenced in this report:

- Throughput matrices (SLURM jobs 5393462, 5393520, 5393524, 5393554): `results/matrix_3B_<jobid>.json`
- ProteinGym, 3B, with per-variant scores (job 5394386): `results/dms_3B_<assay>_5394386.json`
- ProteinGym, 650M (job 5394316): `results/dms_650M_<assay>_5394316.json`
- Full benchmark, per-variant scores, 3B (array 5395036) and 650M (array 5395037): `results/proteingym/<model>_<config>.jsonl.gz`
- Full-benchmark analysis: `results/proteingym/summary_<model>.json` and `summary_<model>_per_assay.csv` (1206 rows: one per assay/configuration)